How Much Must You Use a Local Language Model Before It Pays Off?

Consumer GPUs, open inference engines, and low-cost online tokens

lawyer112

28 October 2026

Abstract With expert offloading, a 12–24 GB gaming GPU plus 64 GB of RAM can now run the 125-billion-parameter Qwen3.8-Flash-Next (the Strata route), while larger DeepSeek models still need one GPU with very large host memory or a 128 GB unified-memory device. This note places these routes in one cost framework with the direct DeepSeek API, Command Code Go/GOAT, and OpenCode Go/Plus, and adds Strata records from one RTX 4090 desktop. In its speed suite, short-chat decoding had a median of 102.4 tokens/s; across 613 real requests with long contexts, medians were only about 32–34 tokens/s, and each engine-busy hour produced about 71,000 output tokens. For this measured workload, 120 busy hours per month cost CNY185 through the direct DeepSeek API if every request were off-peak and CNY370 if every request were at peak, whereas the USD10 GOAT or OpenCode Go fee (about CNY66.77) covers it. A new RTX 5070 desktop costing about CNY16,600 pays back within 36 months only at about 167 busy hours per month, and only for a user who would otherwise pay peak direct prices; against GOAT it needs about 329 hours even if every request is priced at peak. On the same basis, a Mac Studio or DGX Spark needs more than 400 hours, and a 5090 system with 256 GB of RAM cannot reach the threshold within a month. For an existing gaming PC, the cash cost is close to that of a cheap plan, so the choice rests on privacy, control, waiting time, and quality. Answer quality was not evaluated, and the local records cover one machine, one refusal-reduced fine-tune, and about one week of one user's use.

1. Running on consumer hardware is different from recovering its cost

The entry barrier for local language models fell markedly in 2026. A mixture-of-experts model activates only a small share of its parameters for each token, so an inference engine can keep frequently used experts in VRAM and the rest in system RAM or even on an SSD. Strata uses this approach to run the 125-billion-parameter Qwen3.8-Flash-Next on a gaming GPU with 12 GB or more plus 64 GB of RAM; KTransformers and FreeToken run DeepSeek on one GPU with hundreds of gigabytes of host memory; DwarfStar targets 128 GB unified-memory Macs and DGX Spark. [1], [2], [3], [4]

The online side changed too. Several coding tools now turn a USD1 or USD10 monthly fee into model allowances worth several times the fee, and DeepSeek prices requests by time of use, with

peak prices twice the off-peak level. [5], [6], [7] Whether a model can run locally and whether a machine is worth buying are therefore separate questions. This note asks four: what hardware each route needs; how fast it runs in actual use; what the same workload costs online; and how much use a new or existing machine needs before it pays off, together with returns beyond cash.

2. Existing approaches and what this note adds

A common estimate divides a GPU's price by an API price or a rental rate. The price aggregator VRAMGlass, for example, compares buying a 3090 with hourly rental at 4, 8, and 24 hours of daily use, and states that residual value and the rest of the computer are excluded. [8] Such estimates are useful for scale but usually omit three things: the rest of the system, especially memory, whose price rose sharply in 2026; the gap between single-stream benchmarks and long-context agent use, which the FreeToken paper also notes; and the fact that the cheapest online alternative is often a plan with a large allowance rather than an API list price. [3], [9]

This note adds three things: a sourced table of hardware conditions and author-reported speeds for each route; measured throughput and token mix from one 4090 desktop; and a single "engine-busy hour" basis for calculating both online bills and the usage a local machine needs to pay off.

3. Materials and methods

Prices, plans, software, hardware, and exchange rates are treated at the verified source cutoff of 26–27 October 2026. This is a scenario calculation at that cutoff, not a live quotation on the publication date. Changes to service rules require new prices and calculations; the tabulated thresholds should not be taken directly as current purchasing conclusions. Five evidence classes are distinguished: **official documentation** for scope and billing rules; **author reports** for performance under stated conditions; **secondary quotations** for price scale; **local measurements** from one RTX 4090 desktop; and **scenario assumptions** for calculations. The first four have sources or raw records; assumptions are marked in the tables.

The local records have two parts. The first is the speed suite bundled with Strata, run after installation on 12 October 2026: three runs per test, thinking off, temperature zero, speed only. The second is the engine log, which records for each request the prompt tokens, reused tokens, reading time, generated tokens, and generation time, but no content. The analysis window starts with the first engine restart after installation and acceptance testing, at 21:37 on 12 October, and ends with the last start on 20 October: five engine sessions and 613 requests. Earlier installation, acceptance, and client-integration requests are excluded. The parsing script and derived data are described in Section 13.

Answer quality was not evaluated, power was not measured, no other machine was tested, and no paid plan account was used. Prices are page snapshots from 26 October 2026. USD plans are converted at that day's reference rate of CNY6.6774 per USD, excluding payment fees; the direct DeepSeek API uses the RMB prices on its Chinese pricing page. [10] A model, an inference engine, and a hosted service are separate layers: an online Flash offering with a similar name is not automatically equivalent to a particular local quantized file.

4. Local routes: what fits, and how fast it is said to run

4.1 Qwen3.8-Flash-Next: one gaming GPU plus 64 GB of RAM

Qwen3.8-Flash-Next consists of a 125-billion-parameter main model and 51 billion parameters of N-gram embeddings, with about 6 billion parameters active per token; the official documentation notes that the N-gram table can be kept in host memory. [11] The small active share and the large

lookup table explain both why it runs on consumer hardware and why memory layouts differ so much between engines.

Route	Capacity basis	Starting hardware	Author-reported speed
Strata (IQ3_XXS and others)	IQ3_XXS needs 47.0 GB of RAM plus VRAM, Q2_0 37.6 GB; a lookup table of about 29 GB stays on the SSD; 66–76 GB download plus an MTP draft layer of about 6 GB	GPU with 12 GB or more and 64 GB of RAM (32 GB runs only the Coder variant)	RTX 5070 with 64 GB: IQ3_XXS 62 tokens/s in short chat, 49 at 128K context, 1,750 tokens/s prompt reading
Unsloth GGUF with llama.cpp	About 75, 79, 90, and 96–114 GB of total memory at 1–4 bits	96–128 GB class device	Not listed
DwarfStar	Q2 file 137.10 GiB, 41.73 GiB resident; Q4 165.11/69.74 GiB; a 95.37 GiB N-gram table stays on the SSD	64 GB Mac (8K context) to 128 GB Spark	Spark: Q2 22.6, Q4 21.2 tokens/s (8K, teacher-forced)
FreeToken	47.7 GiB N-gram table pinned in host memory	Consumer RTX with large host memory	Not obtained

Sources: project documentation. Speeds are author-reported; this note measured only Strata, on one machine (Section 5). [4], [12], [13], [14]

Strata keeps the most frequently used experts in VRAM, all experts in RAM, and the lookup table on the SSD, and uses an MTP draft layer for speculative decoding, which its author says leaves answers unchanged and speeds output by 1.6–1.8 times. [1] Two points should be read with the speeds. First, from version 0.1.42 Strata on NVIDIA GPUs by default skips experts that rank low in a verification window and are not in VRAM, setting their contribution to zero; against exact computation, the author measured 97.5–98.2% agreement on each token's most likely output. Setting `STRATA_ROUTE_TAIL_SKIP=0` disables this and makes single-GPU decoding somewhat more than 10% slower. [15] Second, by default Strata serves one request at a time; concurrency requires a separate setting. The author also estimates about 100–140 tokens/s for a 24 GB RTX 3090; this is an estimate, not a measurement. [12]

For comparison, the dense Qwen3.8-27B has about 27.8 billion parameters. At exactly 4 bits its bare weights take about 13.9 GB, so llama.cpp can fit it on one 24 GB GPU; it is a much smaller model and not a reduced version of Flash-Next. [16], [17]

4.2 DeepSeek: beyond one GPU, large host memory or SSD streaming

Route and model	Hardware basis	Author-reported speed	Limits
KT-Kernel with SGLang, V4-Flash-0731	Validated setup: one 32 GB 5090, 200 GB+ of RAM, about 340 GB of storage; 3090, 4090, and 5090 architectures marked validated	20+ tokens/s (5090)	16K context, two requests at most, prefix cache off, 60 CPU inference threads
FreeToken, V4-Flash MXFP4	5090; the paper uses rented servers capped at 6–8 threads and one desktop	22–25 tokens/s across four agent workloads	Author results, unreplicated
FastLLM, one GPU plus NUMA	Launch commands provided	No formal throughput recorded	TP8 results cannot be extrapolated
DwarfStar, V4-Flash Q2 (about 81 GiB)	96/128 GB devices	M5 Max: 39.35 at 2K, 36.14 at 16K; Spark: 18.05, 15.10	128-token greedy probe
V4.1-Flash-1017 (the model behind the online API)	DwarfStar Q2: 341 GiB file, 152 GiB main weights; one 128 GB device needs SSD streaming. Strata 0.2.5 needs 245–280 GB of disk	Strata: 4–6 tokens/s on an RTX 5070 with 64 GB, 9–11 on an RTX 3060 with 46 GB	2.00 bpw; author results

Sources: project documentation and the FreeToken paper. [2], [3], [18], [19], [20], [21]

The table has two implications. First, a “single consumer GPU” usually sits beside more than 200 GB of host memory, so the budget cannot count the GPU alone. Second, the V4.1-Flash model behind the online API now runs on a 64 GB gaming PC, but at only a few to roughly ten tokens per second; replacing the online Flash this way buys offline operation and control rather than savings. The FreeToken paper also reports that some engines lose about 30% of their single-turn speed in multi-turn agent workloads and show tail time-to-first-token delays of several hundred seconds. [3]

4.3 How to read these speeds

Prefill (reading the prompt) and decoding (generating tokens one by one) are different measures. The speeds above come from different authors, contexts, and protocols: DwarfStar’s Spark figures use teacher-forced decoding, and Strata’s include speculative decoding and its default expert skipping. They indicate scale; they do not form a controlled leaderboard.

5. Measurements on one RTX 4090 desktop

5.1 Configuration

The machine has an RTX 4090 24 GB, a Ryzen 7 7800X3D, 64 GB of RAM, and Windows 11. It ran Strata 0.1.43 (commit 7695859) with Huihui's refusal-reduced Swift IQ3_XXS build of Qwen3.8-Flash-Next; the two model files total 75,966,073,120 bytes, and their SHA-256 hashes match the pinned release. [22] The configuration used a 256K context, an 8-bit KV cache, 32,768 KV cells kept in VRAM, a 26 GiB RAM budget for experts (the rest read through the file cache), MTP speculative decoding, and one request at a time. Refusal reduction changes refusal behavior, not hardware needs; Swift is a fine-tune that shortens thinking. Neither is the official release. [23]

5.2 Speed suite

Test	Size	Speed (median, range)	Other
Short chat	9 runs of 100 generated tokens	Decode 102.4 (87.4–119.2) tokens/s	First token 0.26 s
Fresh prompt reading	4K/16K/32K	Reading 2,375/3,562/3,663 tokens/s	First token 1.71/4.55/8.78 s
Agent follow-up turns	5 turns, last turn about 14,500 tokens	Decode 119.7 (88.8–158.0) tokens/s	Cache hit 68.6%
Long-context retrieval	255,980 input tokens	Not reported separately	All three positions found; 70–75 s per request

During testing, committed memory peaked at 73.94 GiB against a limit of 79.10 GiB, and available physical memory fell to 8.61 GiB. While serving the model, the computer could hardly take on other heavy work at the same time.

5.3 Actual use

The analysis window contains 613 requests and 5.09 hours of engine-busy time (prompt reading plus generation), 66.2% of it spent decoding. Prompts averaged about 94,000 tokens, of which 90.7% were reused prefixes; 5.35 million tokens were read fresh and 363,000 tokens were generated. Per busy hour, that is about 71,319 output tokens, about 1.05 million freshly read prompt tokens, and about 10.27 million reused prompt tokens. The overall decoding speed was 29.9 tokens/s. The log does not record the caller; prompts this long with this much reuse match the way coding agents resend the whole context on every turn.

Among requests generating at least 64 tokens, prompts of 32K–64K had a median of 31.8 tokens/s (P10–P90 11.6–55.4; 143 requests), and prompts of 64K or more had 33.9 tokens/s (18.3–52.5; 415 requests). The 79 fresh prompts above 16,000 tokens without reuse were read at a median of 1,064 tokens/s, with a median wait of 40.8 s. Median decoding speeds for the five engine sessions were 49.7, 45.9, 29.7, 12.0, and 13.9 tokens/s, and on 20 October the engine reported one 60-second stall while reading a prompt. The causes were not investigated.

5.4 What these figures show

Short-chat benchmarks run at about 100 tokens/s; actual long-context use ran at roughly one third of that, with large differences between sessions. Capacity estimates should use output per busy

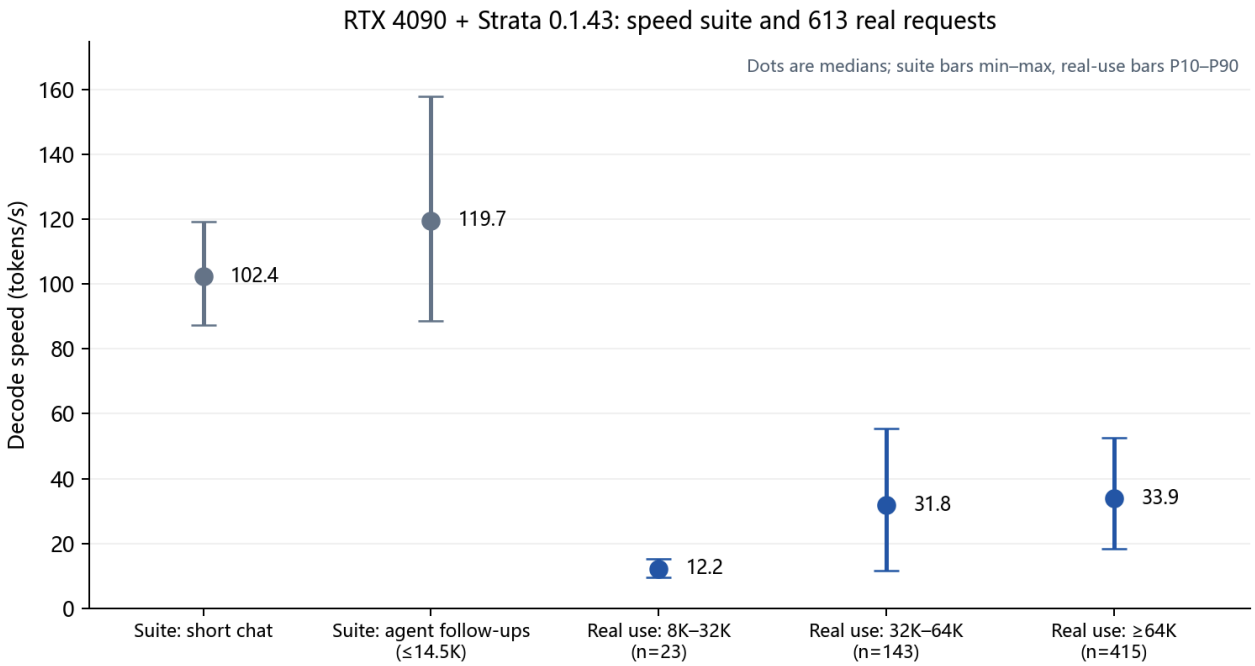


Figure 1: The RTX 4090 with Strata 0.1.43: speed suite and real requests. Dots are medians; suite bars span minimum to maximum and real-use bars span P10 to P90. All 23 requests in the 8K–32K bucket came from the two slow sessions on 20 October.

hour under real workloads rather than benchmark peaks. The records come from one machine, one refusal-reduced fine-tune, and about one week of one user's work. The 26 GiB expert budget chosen to fit a 256K context may be slower than configurations with smaller contexts, but no controlled comparison was made.

6. Hardware prices and system budgets

VRAMGlass aggregates Xianyu listings, takes medians, counts each seller once, and excludes mining cards, accessories, complete systems, and similar items; its figures are asking prices, not sales. [24] The table uses the page snapshot of 19:41 on 26 October 2026 throughout.

Device	Used median, CNY	Official price listed on the page, CNY	Notes
RTX 3090 24 GB	8,500	10,009	2020 launch price; now mostly used
RTX 5070 12 GB	7,586	3,666	GPU in the Strata author's test machine
RTX 5090 32 GB	52,349	13,348	Listings may include the 5090 D / D V2 (24 GB)
Mac Studio M5 Max 128 GB	42,000	Not listed	Exact configuration not verified
DGX Spark 128 GB	44,050	26,703	The new 64 GB version may appear in listings

Sources: VRAMGlass device pages. [8], [25], [26], [27], [28]

Except for the 3090, used prices are well above the listed official prices, a sign of a tight GPU market in 2026 and a reason residual values in three years are hard to predict. Memory prices rose as well. In an Akihabara survey in early October, 64 GB DDR5 desktop kits cost from about JPY98,000, averaging about JPY127,000, or roughly CNY4,200–5,400 at the reference rates; in one server-memory catalogue on 20 September, the median 64 GB DDR4 RDIMM cost USD929 and the DDR5 equivalent USD3,186. [9], [29] These are foreign markets and indicate scale only.

Scenario	Initial investment K, CNY	Basis and route
A: RTX 5070 + 64 GB PC	16,586 (14,586–18,586)	GPU 7,586 plus an allowance of 7,000–11,000 for the other parts; Strata runs Next, matching the author's test machine
B: RTX 5090 + 256 GB system	76,349 (65,349–87,349)	GPU 52,349 plus an allowance of 13,000–35,000; KT or FreeToken runs DeepSeek
C: Mac Studio M5 Max 128 GB	42,000	Complete device; DwarfStar and others
D: DGX Spark 128 GB	44,050	Complete device; the official price of 26,703 is a sensitivity case
E: existing RTX 4090 + 64 GB PC	0	The measured machine; only added operating costs count

None of the other parts were quoted, so they are treated as ranges. A's allowance covers the CPU, motherboard, 64 GB of RAM, power supply, NVMe drive, and case. For B, 256 GB of server memory alone could cost from about CNY7,800 (the cheapest DDR4 modules in stock) to CNY24,800 (the median), and a DDR5 platform would cost more. A 3090 has more VRAM than a 5070, but this note has no Strata measurements for it.

7. Online services: allowances, time of use, and discounts

7.1 One DeepSeek-V4.1-Flash, four ways to pay

Service	Price and allowance	Scope
Direct DeepSeek API	Off-peak, per million tokens: CNY1 for uncached input, CNY0.02 for cache hits, CNY4 for output; peak prices double	Programmatic access; peak hours are 09:00–12:00 and 14:00–18:00 Beijing time on weekdays
Command Code Go	USD1 per month with a USD10 DeepSeek V4.1 Flash allowance	Command Code client only; no API
Command Code GOAT	USD10 per month with a USD80 allowance	Provider API available

Service	Price and allowance	Scope
OpenCode Go	USD10 per month with a USD80 allowance	Coding-agent traffic
OpenCode Go Plus	USD40 per month with a USD160 allowance	Same scope

Sources: provider pricing and plan pages. Plans count usage at the USD list prices, including DeepSeek's peak prices. [5], [6], [7], [30], [31], [32]

Allowances form a **shared pool**. Using USD20 of the DeepSeek allowance and USD7.50 of the Qwen3.8 Flash allowance on OpenCode Go consumes 20/80 + 7.5/30 = 50% of the pool. [6] Short windows can bind before the monthly allowance does: OpenCode allows 20% per five hours and 50% per week; GOAT allows 14 credits per five hours and 35 per week. Command Code's Go documentation conflicts with itself: the plan page states 3 credits per five hours and 6 per week, while the usage table states 2/5; this note leaves the conflict unresolved. Past a window or allowance, Command Code spends top-up credits billed at model prices, and these are not throttled by the windows; OpenCode falls back to the account balance when Extra Usage is enabled. [33], [34]

Billing details also matter. Command Code sells top-ups from five credits; purchased credits roll over while subscription credits expire, so a month's card payment can exceed the cost attributed to that month's use. [35] Terms change often: Command Code Go moved to per-model allowances on 28 September and states that allowances may change at any time. [31]

7.2 "Half price" appears at three different layers

Type	Example	Treatment
Time-of-use price	DeepSeek's off-peak price is half the peak price, and peak hours are weekday daytime	Priced by request time; all-off-peak and all-peak bounds are shown below
Platform deal	Command Code's 50% MiniMax M3 deal applies only when the discounted upstream serves the request	Already built into GOAT's USD47 monthly MiniMax M3 allowance; cannot be applied twice
Vendor introductory price	Gemini 4 Argon's USD2/10 is Google's introductory price until 30 November; from 1 December it is USD4/20	OpenCode labels it 50% off; other channels charge the same; not valid for three years

Sources: provider and model-vendor pricing pages. [6], [7], [34], [36], [37]

For 50 million input and 10 million output tokens, MiniMax M3 costs USD54 at list price and USD27 with the deal (about CNY360.6 and CNY180.3). GOAT's USD47 monthly MiniMax M3 allowance includes the deal, but the page does not say which price it counts: at discounted prices this usage fits within the monthly fee; at list prices it exceeds the allowance by USD7. No paid account was used to check. [34] Google's Batch and Flex tiers are also half price and suit batch work that needs no immediate reply. [37]

8. Online bills for the same workload

Moving the Section 5 workload to the online API means, per busy hour, about 1.05 million uncached input tokens, 10.27 million cache-hit input tokens, and 71,000 output tokens. At DeepSeek's RMB prices, one busy hour costs about CNY1.54 off-peak and CNY3.08 at peak. Off-peak, uncached input accounts for 68.2% of the cost, cache hits for 13.3%, and output for 18.5%.

Busy hours per month (off-peak/ peak, CNY)	Light: 30 h	Medium: 120 h	Heavy: 240 h
Direct DeepSeek	46.25/92.51	185.02/370.03	370.03/740.07
Command Code Go	6.68/32.56	125.22/310.53	310.53/681.17
Command Code GOAT	66.77/66.77	66.77/66.77	66.77/273.84
OpenCode Go	66.77/66.77	66.77/66.77	66.77/273.84
OpenCode Go Plus	267.10/267.10	267.10/267.10	267.10/267.10

Plan cost = monthly fee + max(0, metered cost − model allowance). This is the cost attributed to the month's use, not a guaranteed charge. GOAT's USD80 allowance covers about 346 off-peak or 173 peak busy hours; Go's USD10 covers only about 43 or 22 hours.

Several sensitivities apply. If the online cache-hit rate were 95% instead of the local 90.7%, an off-peak busy hour would cost about CNY1.07. A one-pass document task without context reuse (input five times output, no caching) costs about CNY9 per million output tokens off-peak and CNY18 at peak through the direct API. Using hosted Qwen3.8 Flash as a same-model baseline (whether

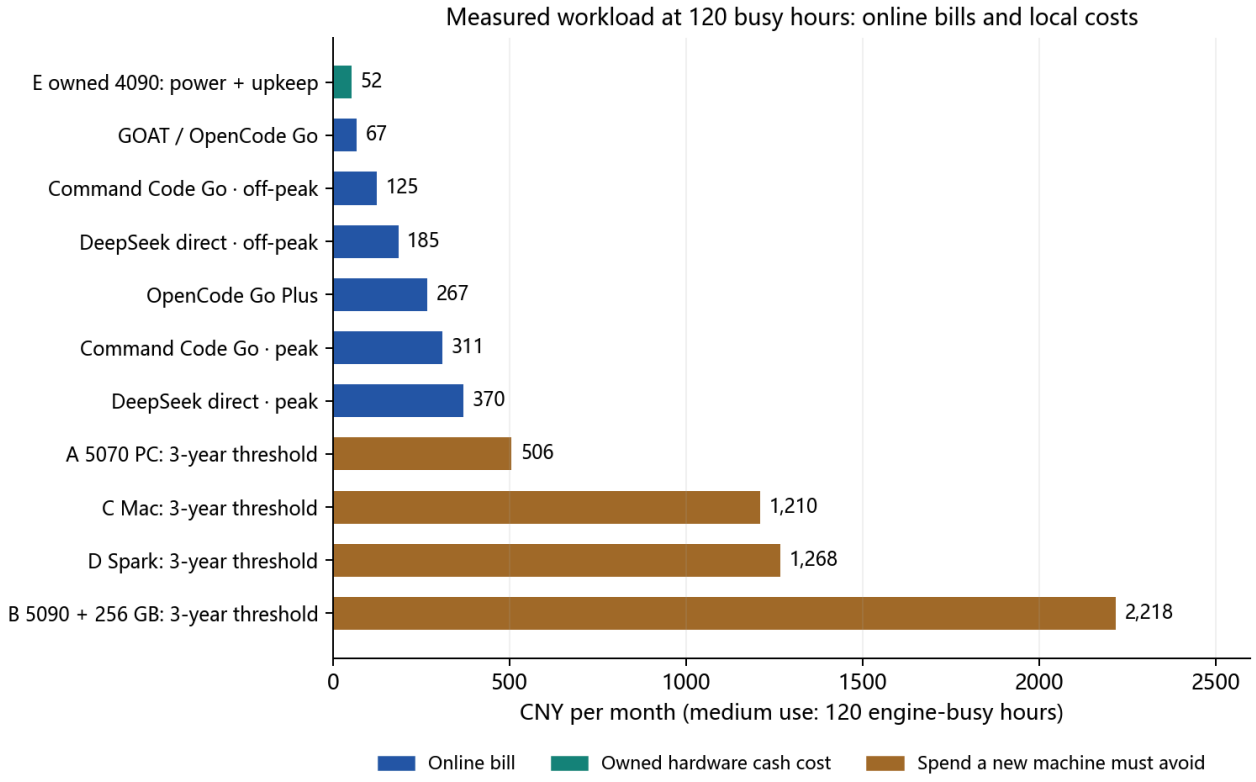


Figure 2: Monthly cost of the measured workload at 120 busy hours: online bills, the cash cost of owned hardware, and the monthly spending a new machine must avoid to pay off within 36 months.

it uses the same weights as the open release is unverified), a busy hour costs about CNY2.37. Different tokenizers also give the same task different token counts.

For this workload, light use of about 30 hours a month is cheapest on Go for work that fits inside the Command Code client. Where an API is needed, the direct API costs about CNY46 off-peak, less than GOAT, and about CNY93 if everything falls at peak, more than GOAT. Personal coding for several hours a day is cheapest on GOAT or OpenCode Go; Plus becomes relevant only far beyond those allowances.

9. Payback: how much use is enough?

9.1 Formulas

Let K be the initial investment, r the residual-value share, and H the busy hours per month. Local operating cost is O = power × H × CNY0.60/kWh + maintenance; electricity price and power are assumptions, and idle standby is excluded. To pay off within 36 months, the online spending actually avoided each month must be at least:

36-month threshold = K × (1 − r) / 36 + O.

With v as the online cost replaced per busy hour (CNY1.54 off-peak and CNY3.08 at peak for direct access), the busy hours needed are:

H* = [K × (1 − r) / 36 + maintenance] / (v − electricity per hour).

Against a plan, each additional hour within the allowance adds nothing to the plan's cost, so local use can save at most the fixed fee; only use beyond the allowance is valued at v. The H* against GOAT is derived this way.

9.2 Results

Scenario	36-month threshold at 120 h, CNY/month	vs direct, peak (h/month)	vs direct, off-peak	vs GOAT, peak
A: 5070 PC	506 (450–561)	167 (148–187)	361	329
B: 5090 + 256 GB	2,218 (1,912–2,523)	806 (693–919)	Over 720	Over 720
C: Mac Studio	1,210	402	Over 720	558
D: DGX Spark	1,268	423	Over 720	580
E: owned 4090	52 (cash cost)	7	16	Already below the fee

Ranges use the low and high allowances for other parts. Busy hours count only the time the engine spends processing requests, not the gaps while the user thinks or tools run, so the machine stays on longer. A month has 720 hours; 240 hours is roughly eight hours of continuous busy time each day. The table assumes every machine matches the 4090′s measured throughput, which is optimistic for A, B, C, and D. Power is assumed at 0.35, 0.65, 0.18, 0.20, and 0.45 kW for A to E.

For A, this means a CNY16,600 RTX 5070 desktop could recover its cost within three years for a user who would otherwise pay peak direct prices for five or six hours of continuous use every day; at 240 hours a month the simple payback is about 24.8 months (21.8–27.8). Against GOAT, however, it needs about 329 busy hours a month, nearly continuous work. If cheap plans were withdrawn or cut back sharply, the columns against direct pricing would apply instead. Session-

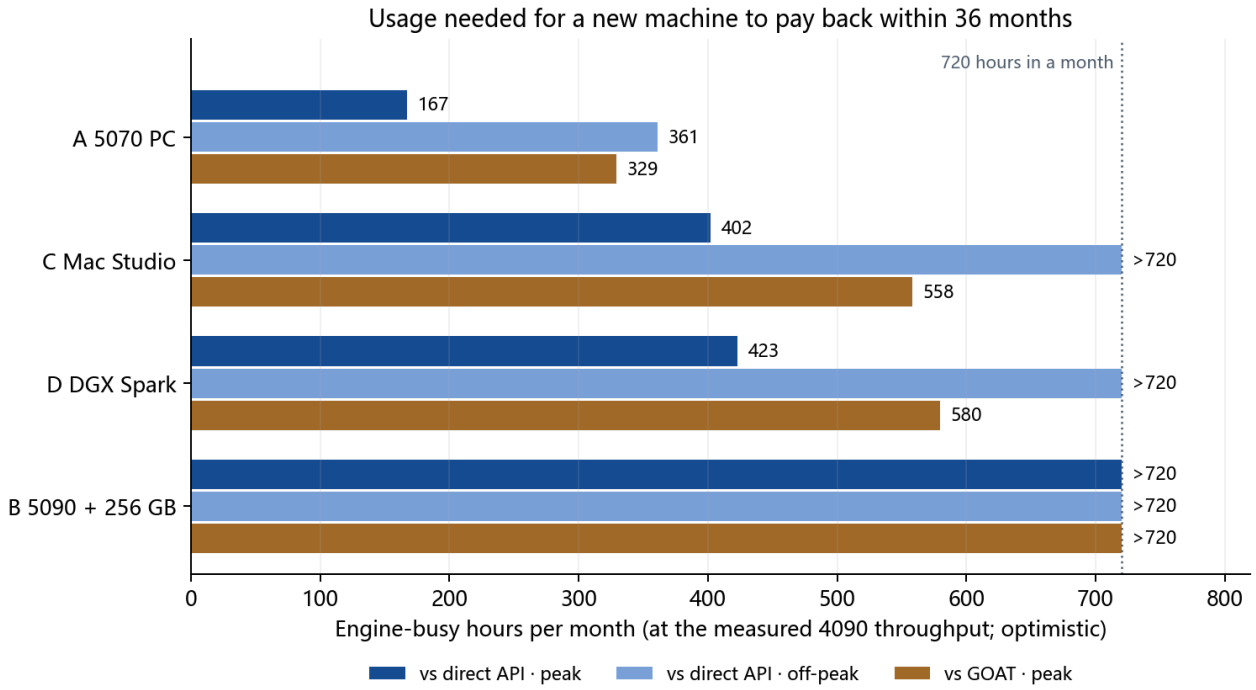


Figure 3: Busy hours per month a new machine needs to pay off within 36 months; values above 720 hours cannot be reached within a month.

to-session differences in throughput change this little: using each of the five sessions' own token mix, A needs 119–195 hours against peak direct pricing.

Residual value lowers the threshold. With a 30% residual value after three years, A, C, and D need about 119, 285, and 299 hours against peak direct pricing; with 50%, about 87, 206, and 217. If a new DGX Spark could be bought at the listed official price of CNY26,703, the threshold at 120 hours would fall to about CNY786 and about 260 hours would be needed against peak direct pricing; whether new units are available at that price in mainland China was not verified.

9.3 Existing hardware

For E only added operating costs count: power and maintenance cost about CNY28/52/85 per month at 30/120/240 busy hours. Against peak direct pricing it saves about CNY64/318/655 a month; against GOAT at peak it saves only about CNY39/14, and about CNY189 only at 240 hours, beyond the allowance. Running a model on an existing gaming PC therefore costs about as much cash as a cheap plan. At the intensity actually observed on this machine over about eight days, only about 19 busy hours a month, use falls in the light band: the machine's cash cost is about CNY25, the direct API about CNY29–59, GOAT CNY66.77, and Go only CNY6.68 for work that fits inside the Command Code client. The real trade-offs lie elsewhere: memory is nearly full while the model runs; fresh prompts above 16,000 tokens take tens of seconds; speed varies between sessions and stalls occur; the default expert skipping is an approximation; and the answer quality of the refusal-reduced fine-tune was not evaluated.

9.4 Replacing an expensive model changes the picture

If a local model could truly replace an expensive one, for example spending of about CNY25.66 per busy hour at Gemini 4 Argon's introductory price, A would pay off at about 19 busy hours per month. This is exactly where quality evaluation is needed: this note has no evidence that a quantized Next can do work otherwise sent to an expensive model.

9.5 Renting a GPU is another route

VRAMGlass lists on-demand rental at CNY1.13 per hour for a 3090, CNY1.7 for a 5070, and CNY3.79 for a 5090, or about CNY136, 204, and 455 for 120 hours; rental is billed by uptime, so actual spending would be higher. [8], [25], [26] Users who need their own or modified weights but not offline operation should compare against rental first. Whether rented instances provide the 64 GB of RAM and fast SSD that Strata needs was not verified.

10. Three different forms of return

The first is **cash not spent**. Only API or subscription fees that would otherwise be paid and can actually be cancelled count as savings. If an expensive online model must be kept anyway, local deployment may simply add a cost.

The second is **time and capability value**, which can be positive or negative. Offline processing, fixed versions, and modifiable weights may make some work easier; installation, maintenance, long-context waiting, and failed retries take time. At CNY50 per hour, ten net hours saved per month are worth CNY500, and ten net hours lost are worth minus CNY500. This is an imputed value, not cash, and it should not be counted again as business profit.

The third is **business contribution**. Suppose a document service contributes CNY2 per accepted task after task-level labor and customer acquisition. To cover A's 36-month threshold of about CNY506 at 120 hours per month requires at least 253 accepted tasks. This note has no evidence that such orders exist or that the machine would complete them on time. The same service could also be delivered through cheap APIs, so the comparison should be the incremental net profit of each option. Renting out an idle GPU is another form of return; market rental prices give an upper bound on the value of a GPU-hour, while platform fees and actual occupancy are unknown.

Check the license conditions of the software and models before deployment: Strata uses the MIT license, but each model and its refusal-reduced or fine-tuned variants carry their own terms. [1]

11. Choosing a route and designing a pilot

Starting situation	Compare first	Verify before buying or migrating
Occasional use, under about 40 busy hours a month	Metered direct API; Go if the work fits its client	Regional and model availability
Daily personal coding	GOAT, OpenCode Go	Tool compatibility, short windows, shared allowances
Existing 12–24 GB GPU and 64 GB of RAM	Strata running Qwen3.8-Flash-Next	Long-context waiting, memory occupation, quality on your own tasks, expert-skip setting
Existing 128 GB unified-memory device	DwarfStar with its specified quantized files	File layout, context, time to first token
Existing 5090 with 192/256 GB of host memory	FreeToken or KT hybrid inference	CPU memory bandwidth, tail time to first token, stability
Need own or modified weights, but not offline use	Compare GPU rental first	Instance memory, disk, and data terms

Starting situation	Compare first	Verify before buying or migrating
Planning a new purchase	Record one month of real bills and busy hours, then apply Section 9	Whether you would otherwise pay peak direct prices, and whether busy hours exceed the payback line

Start with a pilot of 30–50 real tasks; this is a practical starting size, not a claim of statistical sufficiency. Fix weight files, versions, quantization, context, system prompt, and tool settings; record busy hours, measured power, acceptance rate, retries, human review time, and actual charges; then calculate:

Cost per accepted task = (allocated equipment + operation + online calls + maintenance labor) / accepted tasks.

That measure is closer to the purchase decision than cost per million tokens.

12. Limitations and conclusion

Five limitations matter most. First, the local records come from one RTX 4090 desktop, one refusal-reduced Swift fine-tune, and about one week of one user's work; answer quality was not evaluated and power was not measured. Second, the online bills assume that online services see the same token mix as the local machine, while actual tokenization and cache hits will differ. Third, speeds for other routes come from heterogeneous author tests that were not replicated here. Fourth, hardware prices are asking-price medians, other parts were not quoted, and memory prices come from foreign markets; plan rules change often, the Command Code Go window conflict is unresolved, and no paid account was used. Fifth, all local data use one request at a time; batching and concurrency were not tested.

Within these limits, and for a single user sending one request at a time, the supported conclusions are:

- **Against cheap plans, buying new hardware to save token charges rarely pays.** Even the cheapest machine, A, needs about 329 busy hours a month to beat GOAT priced at peak within three years, and about 711 hours if every request is off-peak.
- **Against peak-time direct API pricing, a cheap Strata gaming PC can pay off.** This requires about 167 busy hours a month or more, throughput close to the local records, and a model good enough to substitute. A Mac Studio or DGX Spark needs more than 400 hours, and a 5090 system with 256 GB of RAM cannot reach the threshold within a month.
- **An existing gaming PC costs about as much cash as a cheap plan.** Whether to run locally should depend on privacy, control, waiting time, and quality.
- **Decisions should rest on your own busy hours and real bills,** not on benchmark peaks or an arbitrarily chosen expensive API price.

13. Materials and reproduction

Both language versions share the paper ID and version; each language's webpage and PDF come from the same source. Reproduction has two steps: a parsing script extracts each request's token counts and timings from the raw logs, and a scenario script combines these derived data with the input table (prices, exchange rate, hardware quotes, and assumptions) to produce every result and figure, without starting a model or calling a paid service. The scripts, inputs, derived data, evidence records, glossary, and revision history are kept in the local work package; the default

release contains only the two PDFs and a reader note, and the paper's repository states the scope of any supplementary materials.

The raw records contain local paths and the machine name and are not released. The derived data hold only token counts, timings, and checksums, with no prompt or answer content; any release of them will include the SHA-256 hashes of the raw files. The thread dump from the stall was not copied. Versions 0.1.0 and 0.2.0 are archived and no longer support the current conclusions.

14. AI use

Codex assisted with retrieval, calculation, writing, translation, and checking for versions 0.1.0 and 0.2.0; the backend model was not recorded. Codex also carried out the installation and speed tests on the local machine on 12 October. For version 0.3.0, Kiro assisted with source re-verification, log parsing, recalculation, rewriting both full texts, and checking the webpages and PDFs; its backend model was not recorded either. These checks are AI-assisted self-checks, not independent peer review. The usage log comes from the machine owner's own work and keeps only token counts and timings; there are no participant data, interviews, or records of realized income.

References

[1] Strata Contributors, "Strata: Qwen3.8-Flash-Next on Consumer Hardware." Accessed: Oct. 27, 2026. [Online]. Available: https://github.com/Niko1221/Strata/blob/7695859aa13afb248bdb4d36f9b2e95707db079d/README.zh-CN.md

[2] KTransformers Contributors, "Running DeepSeek-V4-Flash with SGLang and KT-Kernel." Accessed: Oct. 26, 2026. [Online]. Available: https://github.com/kvcache-ai/ktransformers/blob/a694e718e8b3ec2e221277d674e369f3be38d2a4/doc/en/DeepSeek-V4-Flash.md

[3] S. Yang *et al.*, "FreeToken: Efficient Edge-Native MoE Serving with Bandwidth-Adaptive Execution." Accessed: Oct. 27, 2026. [Online]. Available: https://arxiv.org/html/2608.16157v1

[4] S. Sanfilippo, "DwarfStar: Qwen3.8 Flash Next." Accessed: Oct. 26, 2026. [Online]. Available: https://github.com/antirez/ds4/blob/ee7e8405232c8971000cc7ba2b945b00bf6d20d3/docs/QWEN38_FLASH_NEXT.md

[5] Command Code, "GOAT Plan." Accessed: Oct. 26, 2026. [Online]. Available: https://commandcode.ai/docs/plans/goat

[6] OpenCode, "Go." Accessed: Oct. 26, 2026. [Online]. Available: https://opencode.ai/v2/docs/console/go/

[7] DeepSeek, "模型 & 价格." Accessed: Oct. 26, 2026. [Online]. Available: https://api-docs.deepseek.com/zh-cn/quick_start/pricing/

[8] VRAMGlass, "GeForce RTX 3090 价格与二手行情." Accessed: Oct. 26, 2026. [Online]. Available: https://vramglass.com/zh/gpu/rtx-3090

[9] AKIBA PC Hotline!, "DDR5 メモリの値上がりが目立つ、64GB×2 枚組は約 40 万円に急騰 [10 月前半のメモリ価格]." Accessed: Oct. 26, 2026. [Online]. Available: https://akiba-pc.watch.impress.co.jp/docs/price/monthly_repo/2148207.html

[10] ExchangeRate-API, "Latest USD Reference Rates." Accessed: Oct. 27, 2026. [Online]. Available: https://open.er-api.com/v6/latest/USD

[11] Qwen Team, "Qwen3.8-Flash-Next." Accessed: Oct. 27, 2026. [Online]. Available: https://github.com/QwenLM/Qwen3.8-Flash-Next/blob/69885871a64393807d988b27b1b5e380e8f28526/README.md

[12] Strata Contributors, "Models: Sizes, Memory and Speed." Accessed: Oct. 27, 2026. [Online]. Available: https://github.com/Niko1221/Strata/blob/7695859aa13afb248bdb4d36f9b2e95707db079d/docs/MODELS.md

[13] Unsloth, "Qwen3.8-Flash-Next: How to Run Locally." Accessed: Oct. 26, 2026. [Online]. Available: https://unsloth.ai/docs/models/qwen3.8-next

[14] FreeToken Contributors, "FreeToken: Supported Models." Accessed: Oct. 27, 2026. [Online]. Available: https://github.com/FlashML-org/FreeToken/blob/ba31db2ea708b61d7313bccb3ee4adf9d11ecb90/docs/models.md

[15] Strata Contributors, “Details: Missed Experts in a Verify Window.” Accessed: Oct. 27, 2026. [Online]. Available: https://github.com/Niko1221/Strata/blob/7695859aa13afb248bdb4d36f9b2e95707db079d/docs/DETAILS.md

[16] Qwen Team, “Qwen3.8-27B Model Card.” Accessed: Oct. 26, 2026. [Online]. Available: https://huggingface.co/Qwen/Qwen3.8-27B

[17] ggml-org Contributors, “llama.cpp.” Accessed: Oct. 27, 2026. [Online]. Available: https://github.com/ggml-org/llama.cpp/tree/caa8191df159f764fb0160e3d2fc6be008948aee

[18] FastLLM Contributors, “DeepSeek-V4 / DeepSeek-V4-Flash 部署指南.” Accessed: Oct. 27, 2026. [Online]. Available: https://github.com/ztxz16/fastllm/blob/3d732fdbbaaf2360aafb9f9d529fdede4abc8b0e/docs/deepseek.md

[19] S. Sanfilippo, “DwarfStar: Models and Vision.” Accessed: Oct. 26, 2026. [Online]. Available: https://github.com/antirez/ds4/blob/ee7e8405232c8971000cc7ba2b945b00bf6d20d3/docs/MODELS.md

[20] S. Sanfilippo, “DwarfStar: Performance and Benchmarking.” Accessed: Oct. 26, 2026. [Online]. Available: https://github.com/antirez/ds4/blob/ee7e8405232c8971000cc7ba2b945b00bf6d20d3/docs/PERFORMANCE.md

[21] Strata Contributors, “Strata README: DeepSeekV4.1-Flash-1017.” Accessed: Oct. 27, 2026. [Online]. Available: https://github.com/Niko1221/Strata/blob/4b9e05a2a806a3fe24f7b2560d98626c37dade35/README.md

[22] huihui-ai, “Huihui-Qwen3.8-Flash-Next-abliterated-GGUF: swift-IQ3_XXS.” Accessed: Oct. 27, 2026. [Online]. Available: https://huggingface.co/huihui-ai/Huihui-Qwen3.8-Flash-Next-abliterated-GGUF/tree/4fdf4dc4d85184e2a51f5d33d7373ce053b3ec60/swift-IQ3_XXS

[23] P. E. Weidmann, “Heretic.” Accessed: Oct. 27, 2026. [Online]. Available: https://github.com/p-e-w/heretic/tree/80f81c90be92ab507d310fe4cb5f39745e6d357c

[24] VRAMGlass, “数据方法.” Accessed: Oct. 26, 2026. [Online]. Available: https://vramglass.com/zh/methodology

[25] VRAMGlass, “GeForce RTX 5070 价格与二手行情.” Accessed: Oct. 27, 2026. [Online]. Available: https://vramglass.com/zh/gpu/rtx-5070

[26] VRAMGlass, “GeForce RTX 5090 价格与二手行情.” Accessed: Oct. 26, 2026. [Online]. Available: https://vramglass.com/zh/gpu/rtx-5090

[27] VRAMGlass, “Mac Studio M5 Max 128GB 价格与二手行情.” Accessed: Oct. 26, 2026. [Online]. Available: https://vramglass.com/zh/gpu/mac-studio-m5-max-128gb

[28] VRAMGlass, “DGX Spark 128GB 价格与二手行情.” Accessed: Oct. 26, 2026. [Online]. Available: https://vramglass.com/zh/gpu/dgx-spark

[29] MemoryClearance, “DDR4 or DDR5 for a Server Refresh in the 2026 Shortage.” Accessed: Oct. 26, 2026. [Online]. Available: https://www.memoryclearance.com/blog/ddr4-or-ddr5-server-refresh-2026.html

[30] DeepSeek, “Models & Pricing.” Accessed: Oct. 26, 2026. [Online]. Available: https://api-docs.deepseek.com/quick_start/pricing/

[31] Command Code, “Go Plan.” Accessed: Oct. 26, 2026. [Online]. Available: https://commandcode.ai/docs/plans/go

[32] OpenCode, “Low Cost Coding Models for Everyone.” Accessed: Oct. 26, 2026. [Online]. Available: https://opencode.ai/go

[33] Command Code, “Usage Limits.” Accessed: Oct. 26, 2026. [Online]. Available: https://commandcode.ai/docs/resources/usage-limits

[34] Command Code, “Pricing and Limits.” Accessed: Oct. 26, 2026. [Online]. Available: https://commandcode.ai/docs/resources/pricing-limits

[35] Command Code, “Frequently Asked Questions.” Accessed: Oct. 26, 2026. [Online]. Available: https://commandcode.ai/docs/troubleshooting/faqs

[36] Google, “Gemini Developer API Pricing.” Accessed: Oct. 26, 2026. [Online]. Available: https://ai.google.dev/gemini-api/docs/pricing

[37] LiteLLM, “Gemini 4 Argon API Pricing.” Accessed: Oct. 26, 2026. [Online]. Available: https://models.litellm.ai/models/gemini-4-argon